

Name : Kasak Patel

Sec-Batch : C2-B1

Roll No : 06

ML Lab 1

Lab-1 : To study and apply Data Preprocessing techniques on given dataset. Prepare a titanic dataset for training with machine learning algorithm by applying suitable data preprocessing techniques.

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

df = pd.read_csv("titanic.csv")
```

The head() function displays the first five rows of the Titanic dataset. It provides a quick overview of the data and column names. It helps verify that the dataset has been loaded correctly. This is the first step in understanding and exploring the dataset.

```
df.head()
```

	PassengerId	Survived	Pclass	\		Name	Sex	Age
0	1	0	3					
1	2	1	1					
2	3	1	3					
3	4	1	1					
4	5	0	3					

	SibSp	\	Name	Sex	Age
0			Braund, Mr. Owen Harris	male	22.0
1					
1	1		Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
1					
2			Heikkinen, Miss. Laina	female	26.0
0					
3			Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
1					

4			Allen, Mr. William Henry	male	35.0
0					
	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

The `tail()` function displays the last five rows of the Titanic dataset. It helps verify the end of the dataset and check the data structure. This is useful for confirming that the dataset has been loaded completely and correctly.

```
df.tail()
```

	PassengerId	Survived	Pclass	
Name \				
886	887	0	2	Montvila, Rev. Juozas
887	888	1	1	Graham, Miss. Margaret Edith
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"
889	890	1	1	Behr, Mr. Karl Howell
890	891	0	3	Dooley, Mr. Patrick

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	male	27.0	0	0	211536	13.00	NaN	S
887	female	19.0	0	0	112053	30.00	B42	S
888	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	male	26.0	0	0	111369	30.00	C148	C
890	male	32.0	0	0	370376	7.75	NaN	Q

The `describe()` function generates summary statistics for the numerical columns in the dataset. It displays values such as count, mean, standard deviation, minimum, maximum, and quartiles. This helps understand the overall distribution of the data and identify possible outliers.

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	\
count	891.000000	891.000000	891.000000	714.000000	891.000000	
mean	446.000000	0.383838	2.308642	29.699118	0.523008	
std	257.353842	0.486592	0.836071	14.526497	1.102743	
min	1.000000	0.000000	1.000000	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	

75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

The info() function displays the dataset structure, including column names, data types, and the number of non-null values. It helps identify missing values and understand the data before preprocessing.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   PassengerId     891 non-null   int64  
 1   Survived        891 non-null   int64  
 2   Pclass         891 non-null   int64  
 3   Name            891 non-null   object  
 4   Sex             891 non-null   object  
 5   Age             714 non-null   float64 
 6   SibSp           891 non-null   int64  
 7   Parch           891 non-null   int64  
 8   Ticket          891 non-null   object  
 9   Fare            891 non-null   float64 
10   Cabin           204 non-null   object  
11   Embarked        889 non-null   object  
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

The isnull().sum() function counts the missing values in each column. It helps identify which columns require data cleaning before training a machine learning model.

```
df.isnull().sum()

PassengerId    0
Survived        0
Pclass         0
Name            0
Sex             0
Age            177
```

```
SibSp      0
Parch      0
Ticket     0
Fare       0
Cabin     687
Embarked    2
dtype: int64
```

The `unique()` function displays all the distinct values present in the `Embarked` column. It helps identify the different embarkation ports and detect any missing values. This information is useful before encoding categorical data.

```
df["Embarked"].unique()

array(['S', 'C', 'Q', nan], dtype=object)
```

The `replace()` function converts the categorical values in the `Embarked` column into numerical values. Here, S is replaced with 0, C with 1, and Q with 2. This encoding makes the data suitable for machine learning algorithms, which require numerical inputs.

```
df["Embarked"] = df["Embarked"].replace("S",0)
df["Embarked"] = df["Embarked"].replace("C",1)
df["Embarked"] = df["Embarked"].replace("Q",2)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_22936\1861778179.py:3:
FutureWarning: Downcasting behavior in `replace` is deprecated and
will be removed in a future version. To retain the old behavior,
explicitly call `result.infer_objects(copy=False)`. To opt-in to the
future behavior, set `pd.set_option('future.no_silent_downcasting',
True)`
```

```
df["Embarked"] = df["Embarked"].replace("Q",2)
```

```
df.head()
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	SibSp	\	Name	Sex	Age
0			Braund, Mr. Owen Harris	male	22.0
1					
1	1		Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
1					
2			Heikkinen, Miss. Laina	female	26.0
0					

```

3      Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0
1
4      Allen, Mr. William Henry  male  35.0
0

```

```

Parch      Ticket      Fare Cabin Embarked
0      0      A/5 21171   7.2500  NaN     0.0
1      0      PC 17599  71.2833   C85     1.0
2      0  STON/O2. 3101282   7.9250  NaN     0.0
3      0      113803  53.1000  C123     0.0
4      0      373450   8.0500  NaN     0.0

```

```

df["Sex"] = df["Sex"].replace("Male",0)
df["Sex"] = df["Sex"].replace("Female",1)

```

```
df.head()
```

```

PassengerId  Survived  Pclass  \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

```

```

SibSp  \
0      Braund, Mr. Owen Harris  male  22.0
1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0
1
2      Heikkinen, Miss. Laina  female  26.0
0
3      Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0
1
4      Allen, Mr. William Henry  male  35.0
0

```

```

Parch      Ticket      Fare Cabin Embarked
0      0      A/5 21171   7.2500  NaN     0.0
1      0      PC 17599  71.2833   C85     1.0
2      0  STON/O2. 3101282   7.9250  NaN     0.0
3      0      113803  53.1000  C123     0.0
4      0      373450   8.0500  NaN     0.0

```

The `fillna()` function replaces missing values in the Age column with the median age. The median is preferred because it is less affected by extreme values (outliers). This ensures that the dataset has no missing values in the Age column before model training.

```
df["Age"].fillna(df["Age"].median(), inplace=True)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_22936\4235138809.py:1:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["Age"].fillna(df["Age"].median(), inplace=True)
```

```
df["Embarked"].fillna(df["Embarked"].mode()[0],inplace=True)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_22936\548196730.py:1:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["Embarked"].fillna(df["Embarked"].mode()[0],inplace=True)
```

```
df.isnull().sum()           #No missing value in Age n Embarked
```

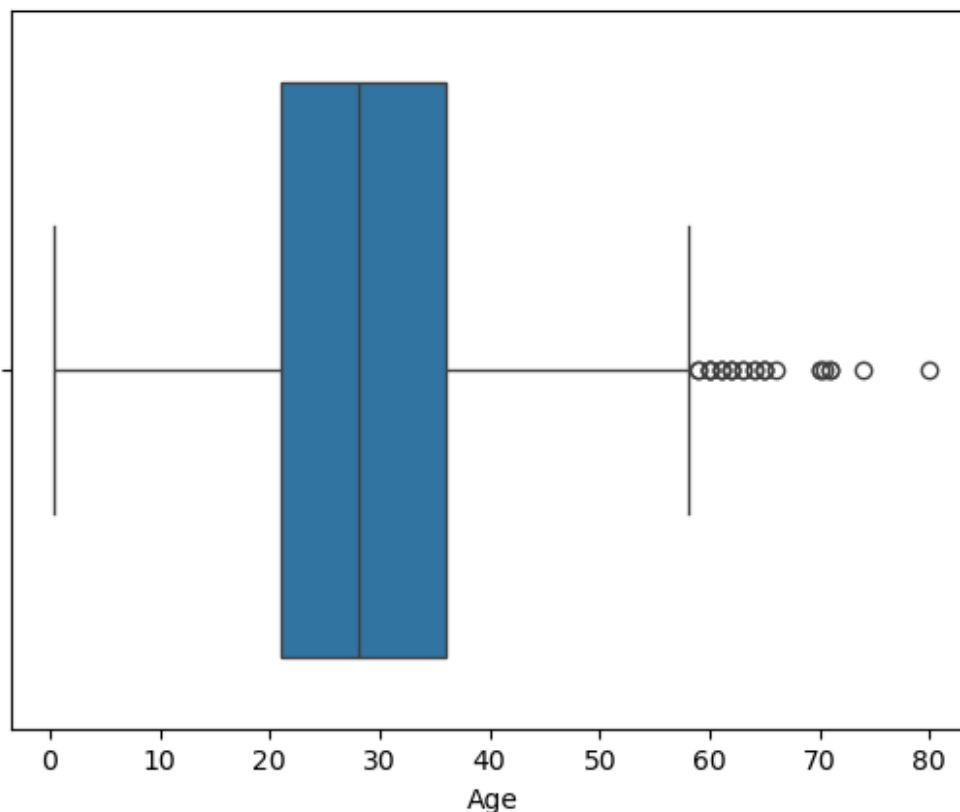
```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch           0
Ticket           0
Fare             0
Cabin            687
Embarked         0
dtype: int64
```

The `drop()` function removes the Cabin column from the dataset. This column is dropped because it contains a large number of missing values. Removing it helps simplify the dataset and improves the quality of data for machine learning.

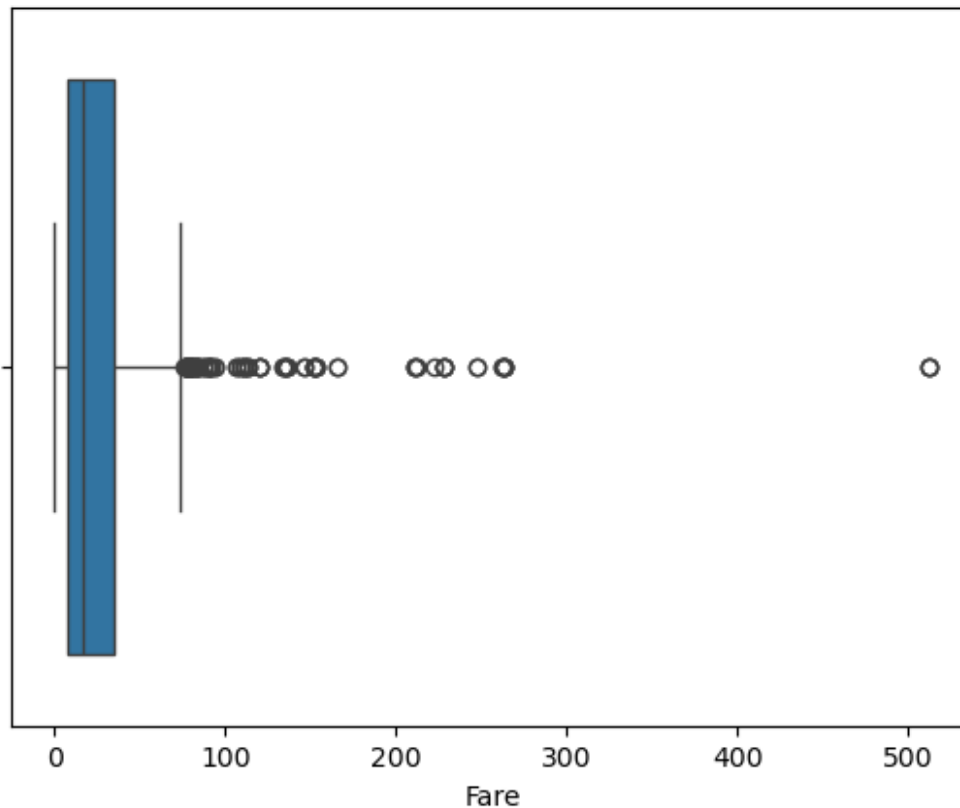
```
df.drop("Cabin",axis=1,inplace=True)
df.drop("PassengerId",axis=1,inplace=True)
df.drop("Ticket",axis=1,inplace=True)
df.drop("Name",axis=1,inplace=True)
df.duplicated().sum()
119
df.drop_duplicates(inplace=True)
```

The `boxplot()` function visualizes the distribution of the Age column. It helps identify outliers, the median, and the spread of the data. This plot is useful for understanding the age distribution before applying machine learning techniques.

```
sns.boxplot(x=df["Age"])
plt.show()
```

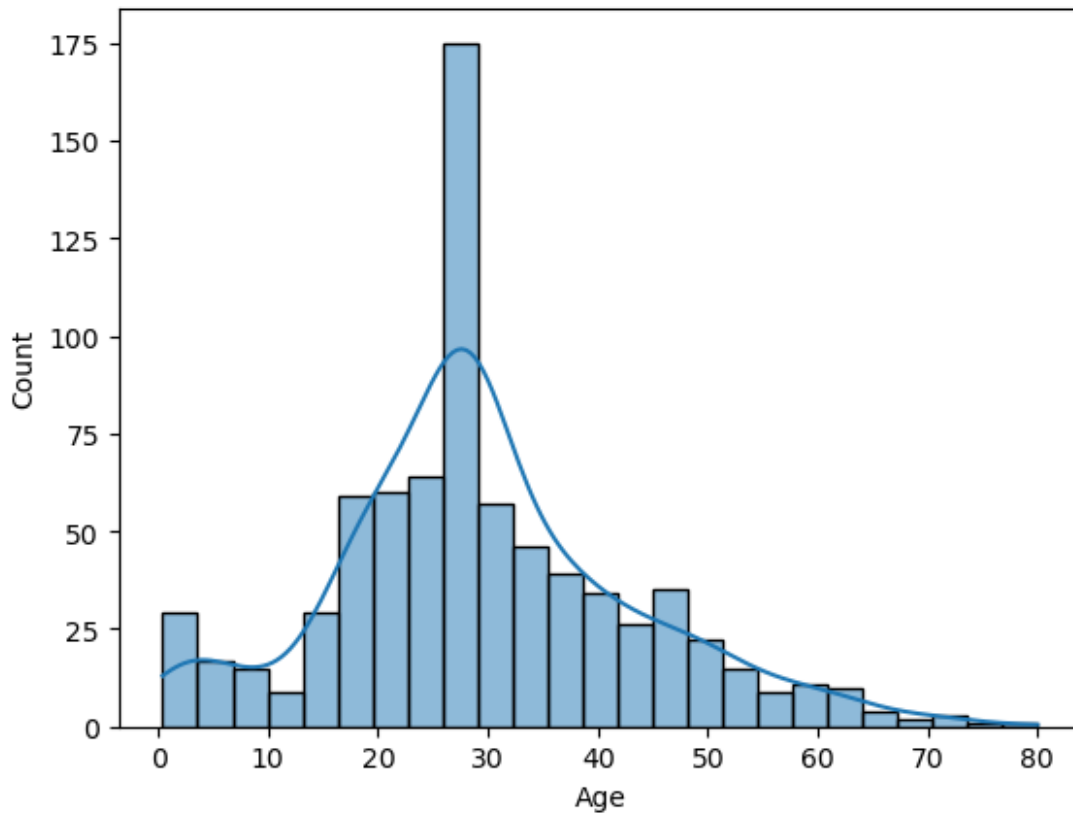


```
sns.boxplot(x=df["Fare"])  
plt.show()
```



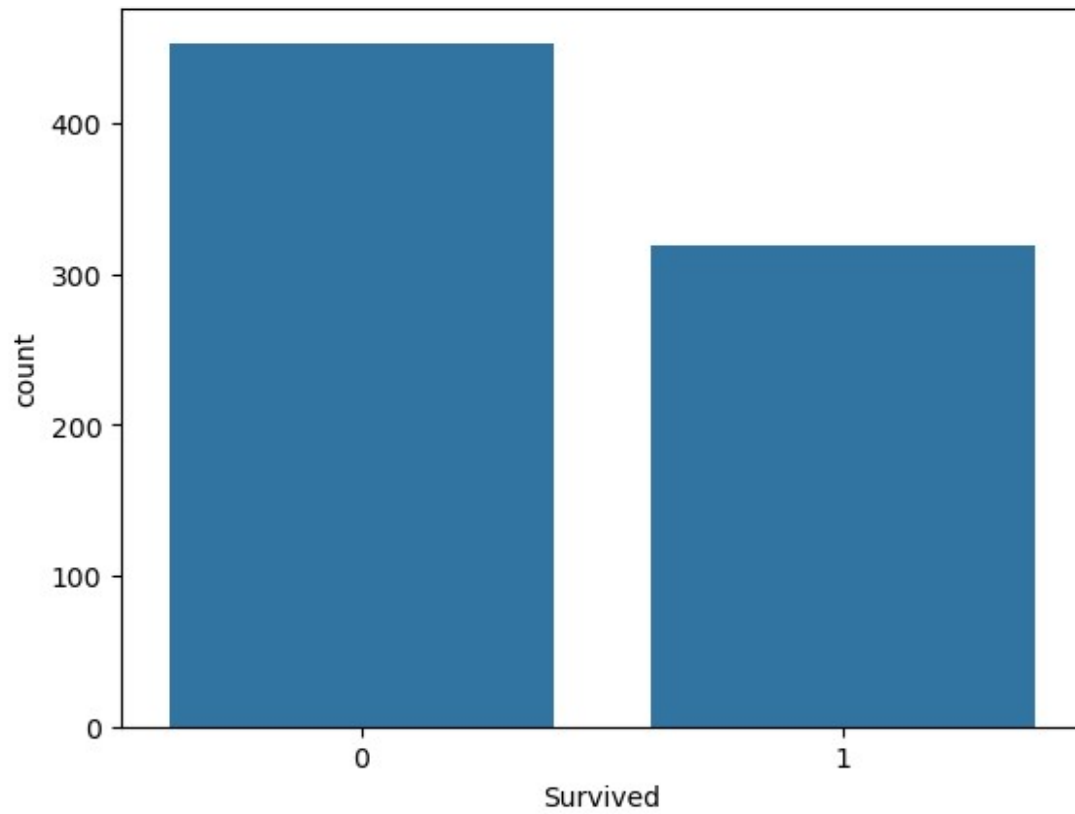
The `get_dummies()` function performs One-Hot Encoding on the Embarked column. It converts each category into a separate binary (0 or 1) column. This makes the categorical data suitable for machine learning algorithms without implying any order between the categories.

```
df = pd.get_dummies(df, columns=["Embarked"])  
sns.histplot(df["Age"], kde=True)  
plt.show()
```

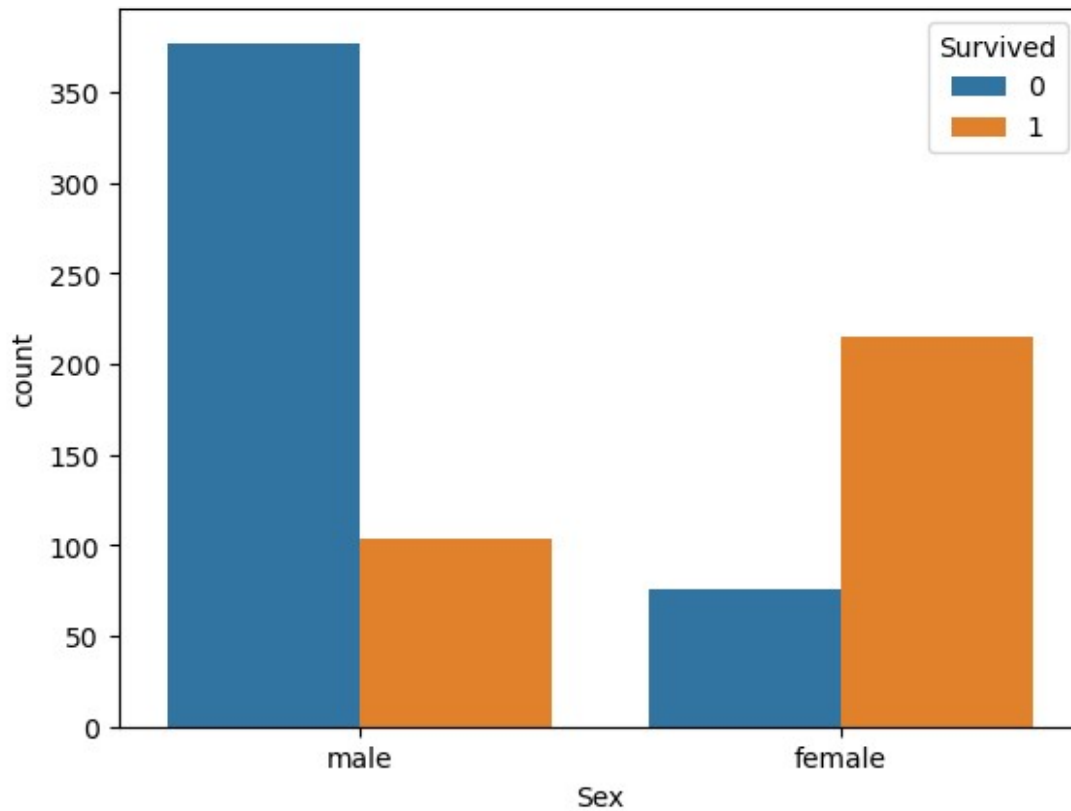



The `countplot()` function displays the number of passengers in each Survived category. It helps visualize how many passengers survived and how many did not. This graph provides a quick understanding of the distribution of the target variable in the dataset.

```
sns.countplot(x="Survived",data=df)           #feature is survived  
plt.show()
```

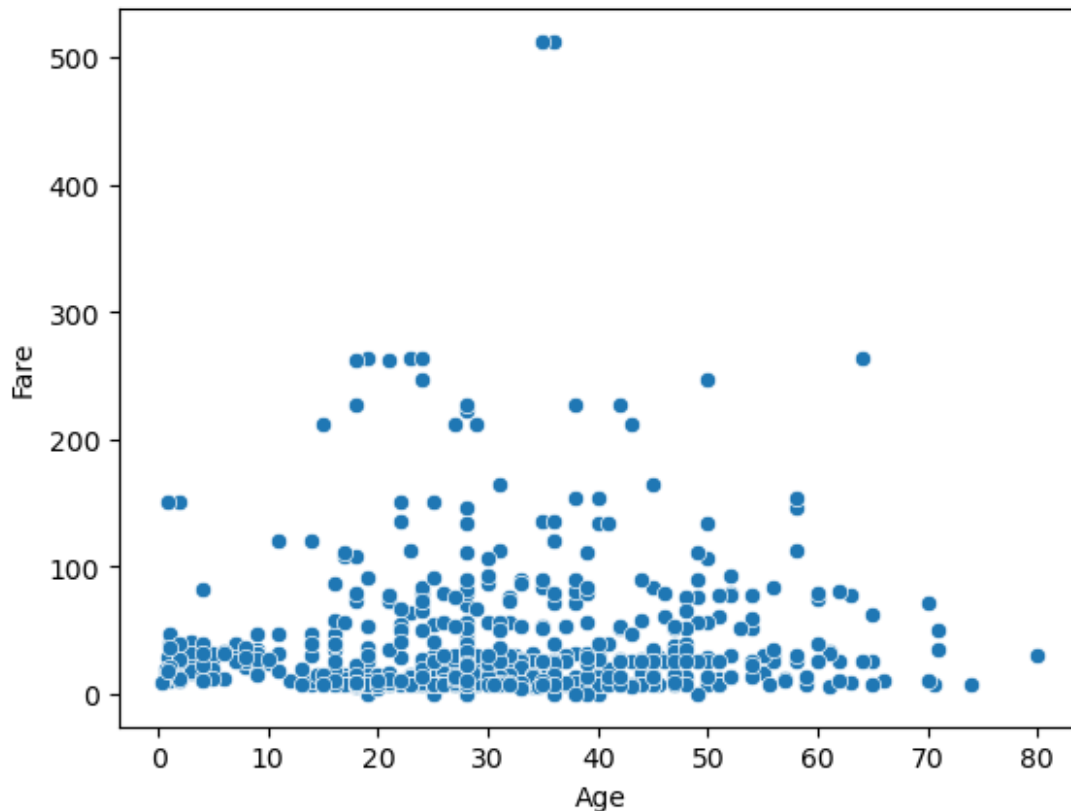


```
sns.countplot(x="Sex", hue="Survived", data=df)  
plt.show()
```



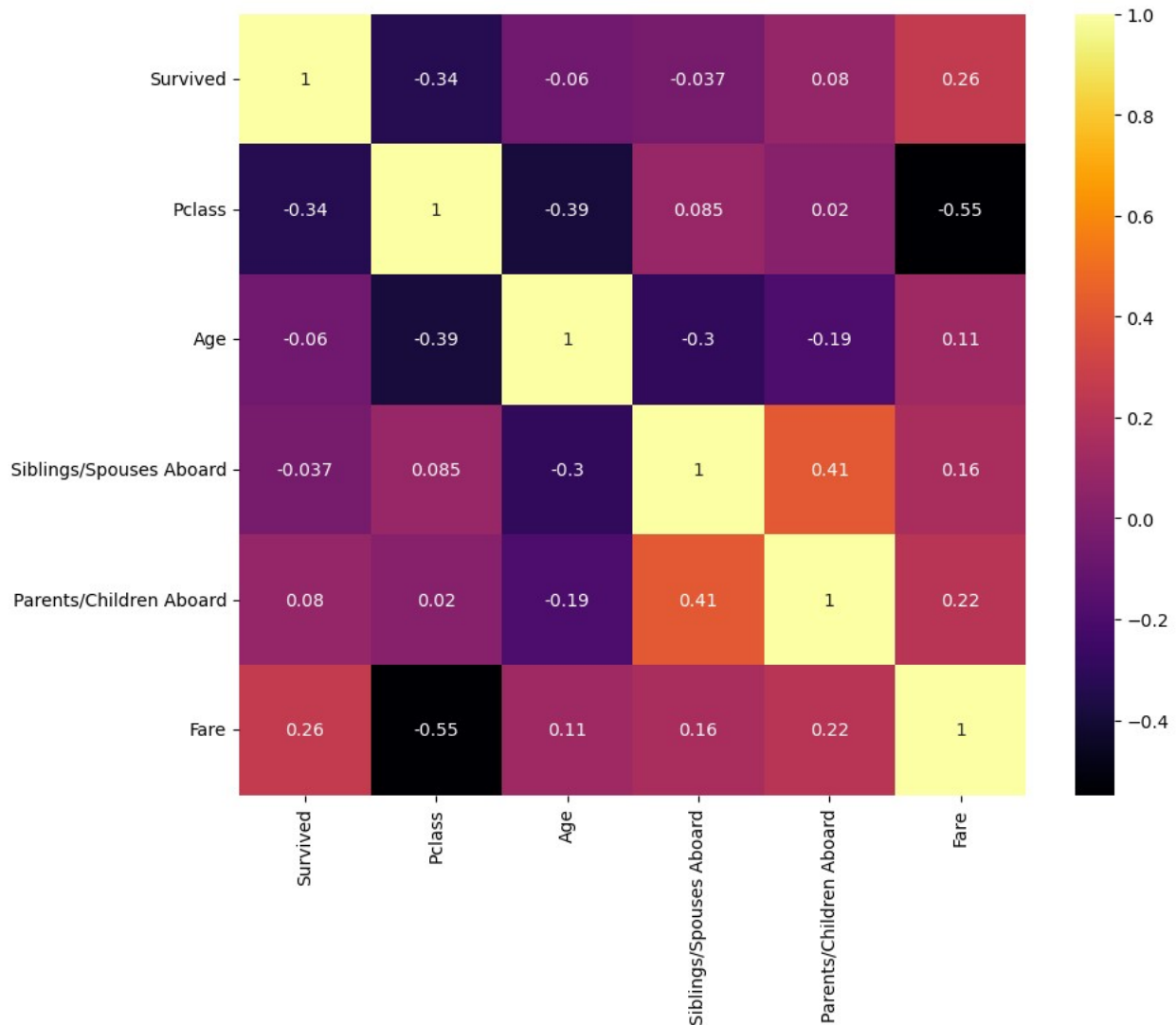
The scatterplot() function shows the relationship between the Age and Fare columns. Each point represents a passenger in the dataset. It helps identify patterns, trends, or outliers between age and ticket fare.

```
sns.scatterplot(x="Age",y="Fare",data=df)  
plt.show()
```



The heatmap() function displays the correlation between numerical features in the dataset. It helps identify positive and negative relationships among variables. This visualization is useful for understanding feature dependencies before training a machine learning model.

```
plt.figure(figsize=(10, 8))  
  
numeric_df = df.select_dtypes(include=['number'])  
  
sns.heatmap(numeric_df.corr(), annot=True, cmap='inferno')  
  
plt.show()
```



```
plt.figure(figsize=(6,4))
sns.countplot(x="Embarked",data=df)

plt.title("Passengers by Embarked Port")
plt.show()
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
```

```
Cell In[13], line 2
```

```
1 plt.figure(figsize=(6,4))
----> 2 sns.countplot(x="Embarked",data=df)
      4 plt.title("Passengers by Embarked Port")
      5 plt.show()
```

```
File ~\anaconda3\Lib\site-packages\seaborn\categorical.py:2631, in
```

```

countplot(data, x, y, hue, order, hue_order, orient, color, palette,
saturation, fill, hue_norm, stat, width, dodge, gap, log_scale,
native_scale, formatter, legend, ax, **kwargs)
    2628 elif x is not None and y is not None:
    2629     raise TypeError("Cannot pass values for both `x` and
`y`.")
-> 2631 p = _CategoricalAggPlotter(
    2632     data=data,
    2633     variables=dict(x=x, y=y, hue=hue),
    2634     order=order,
    2635     orient=orient,
    2636     color=color,
    2637     legend=legend,
    2638 )
    2640 if ax is None:
    2641     ax = plt.gca()

```

File ~\anaconda3\Lib\site-packages\seaborn\categorical.py:67, in
_CategoricalPlotter.__init__(self, data, variables, order, orient,
require_numeric, color, legend)

```

    56 def __init__(
    57     self,
    58     data=None,
    (... )
    64     legend="auto",
    65 ):
---> 67     super().__init__(data=data, variables=variables)
    69     # This method takes care of some bookkeeping that is
necessary because the
    70     # original categorical plots (prior to the 2021 refactor)
had some rules that
    71     # don't fit exactly into VectorPlotter logic. It may be
wise to have a second
    (... )
    76     # default VectorPlotter rules. If we do decide to make
orient part of the
    77     # _base variable assignment, we'll want to figure out how
to express that.
    78     if self.input_format == "wide" and orient in ["h", "y"]:

```

File ~\anaconda3\Lib\site-packages\seaborn_base.py:634, in
VectorPlotter.__init__(self, data, variables)

```

    629 # var_ordered is relevant only for categorical axis variables,
and may
    630 # be better handled by an internal axis information object
that tracks
    631 # such information and is set up by the scale_* methods. The
analogous
    632 # information for numeric axes would be information about log

```

```
scales.
633 self._var_ordered = {"x": False, "y": False} # alt., used
DefaultDict
--> 634 self.assign_variables(data, variables)
636 # TODO Lots of tests assume that these are called to
initialize the
637 # mappings to default values on class initialization. I'd
prefer to
638 # move away from that and only have a mapping when explicitly
called.
639 for var in ["hue", "size", "style"]:
```

```
File ~\anaconda3\Lib\site-packages\seaborn\_base.py:679, in
VectorPlotter.assign_variables(self, data, variables)
674 else:
675     # When dealing with long-form input, use the newer
PlotData
676     # object (internal but introduced for the objects
interface)
677     # to centralize / standardize data consumption logic.
678     self.input_format = "long"
--> 679     plot_data = PlotData(data, variables)
680     frame = plot_data.frame
681     names = plot_data.names
```

```
File ~\anaconda3\Lib\site-packages\seaborn\_core\data.py:58, in
PlotData.__init__(self, data, variables)
51 def __init__(
52     self,
53     data: DataSource,
54     variables: dict[str, VariableSpec],
55 ):
57     data = handle_data_source(data)
--> 58     frame, names, ids = self._assign_variables(data,
variables)
60     self.frame = frame
61     self.names = names
```

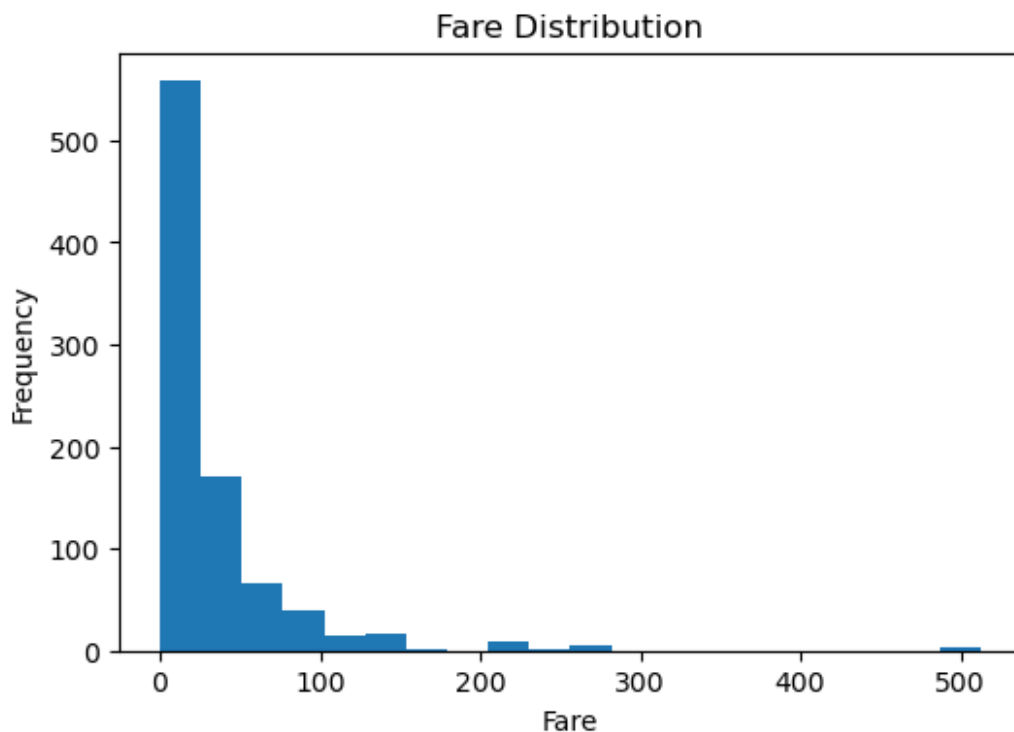
```
File ~\anaconda3\Lib\site-packages\seaborn\_core\data.py:232, in
PlotData._assign_variables(self, data, variables)
230     else:
231         err += "An entry with this name does not appear in
`data`."
--> 232     raise ValueError(err)
234 else:
235
236     # Otherwise, assume the value somehow represents data
237
238     # Ignore empty data structures
239     if isinstance(val, Sized) and len(val) == 0:
```

```
ValueError: Could not interpret value `Embarked` for `x`. An entry with this name does not appear in `data`.
```

```
<Figure size 600x400 with 0 Axes>
```

The `hist()` function displays the distribution of the Fare column. It shows how passenger fares are spread across different ranges. This helps identify the frequency, distribution pattern, and possible outliers in the fare data.

```
plt.figure(figsize=(6,4))
plt.hist(df['Fare'], bins=20)
plt.title('Fare Distribution')
plt.xlabel('Fare')
plt.ylabel('Frequency')
plt.show()
```



```
# Feature Scaling
x = df.drop("Survived", axis = 1)
y = df["Survived"]
```


The StandardScaler is used to standardize the Age and Fare columns. It transforms the values so that they have a mean of 0 and a standard deviation of 1. This improves the performance of many machine learning algorithms by ensuring that numerical features are on a similar scale.

```
# Standard Scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

x[["Age", "Fare"]] = scaler.fit_transform(x[["Age", "Fare"]])

x.head(15)
```

	Pclass	Name	Sex
0	3	Mr. Owen Harris Braund	male
1	1	Mrs. John Bradley (Florence Briggs Thayer) Cum...	female
2	3	Miss. Laina Heikkinen	female
3	1	Mrs. Jacques Heath (Lily May Peel) Futrelle	female
4	3	Mr. William Henry Allen	male
5	3	Mr. James Moran	male
6	1	Mr. Timothy J McCarthy	male
7	3	Master. Gosta Leonard Palsson	male
8	3	Mrs. Oscar W (Elisabeth Vilhelmina Berg) Johnson	female
9	2	Mrs. Nicholas (Adele Achem) Nasser	female
10	3	Miss. Marguerite Rut Sandstrom	female
11	1	Miss. Elizabeth Bonnell	female
12	3	Mr. William Henry Saundercock	male
13	3	Mr. Anders Johan Andersson	male
14	3	Miss. Hulda Amanda Adolfina Vestrom	female
	Age	Siblings/Spouses Aboard	Parents/Children Aboard
Fare			
0	-0.529366	1	0 -
	0.503586		
1	0.604265	1	0

0.783412		
2 -0.245958	0	0 -
0.490020		
3 0.391709	1	0
0.417948		
4 0.391709	0	0 -
0.487507		
5 -0.175106	0	0 -
0.479301		
6 1.737895	0	0
0.393076		
7 -1.946404	3	1 -
0.225719		
8 -0.175106	0	2 -
0.425536		
9 -1.096181	1	0 -
0.044913		
10 -1.804700	1	1 -
0.313652		
11 2.021303	0	0 -
0.115678		
12 -0.671070	0	0 -
0.487507		
13 0.675116	1	5 -
0.020710		
14 -1.096181	0	0 -
0.491443		

The `train_test_split` function is imported to divide the dataset into training and testing sets. The training set is used to train the machine learning model, while the testing set is used to evaluate its performance on unseen data.

```
# Train Test Split
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size =
0.20, random_state = 42)

print("Training data :", x_train.shape)
Training data : (709, 7)

print("Texting data :", x_test.shape)
Texting data : (178, 7)

print(x_train.head())
```

	Pclass	Name	Sex	Age	\
730	2	Mr. Moses Aaron Troupiansky	male	-0.458514	
390	3	Mr. Johan Birger Gustafsson	male	-0.104255	

118	3	Miss. Ellis Anna Maria Andersson	female	-1.946404
440	2	Ms. Encarnacion Reynaldo	female	-0.104255
309	1	Miss. Emily Borie Ryerson	female	-0.812774

	Siblings/Spouses Aboard	Parents/Children Aboard	Fare
730	0	0	-0.388018
390	2	0	-0.490020
118	4	2	-0.020710
440	0	0	-0.388018
309	2	2	4.624145

```
import warnings

warnings.filterwarnings("ignore")
```

Viva Questions with Answers

Q1. What is data preprocessing?

Answer: Data preprocessing is the process of cleaning and transforming raw data into a suitable format for machine learning.

Q2. Why do we handle missing values?

Answer: Missing values can reduce model accuracy and cause errors during training. Therefore, they are filled or removed.

Q3. What is Label Encoding?

Answer: Label Encoding converts categorical text values into numerical values so that machine learning algorithms can process them.

Q4. Why is feature scaling required?

Answer: Feature scaling ensures all numerical features have a similar range, improving the performance of many machine learning algorithms.

Q5. Why was the Cabin column removed?

Answer: The Cabin column contained too many missing values, making it less useful for model training.